

0.1 Introduction

The reachability toolkit focuses on network survivability and connectivity probability. It computes the probability that at least one active source-to-sink route survives random component failures in a given DAG network.

For some infrastructure reliability problems, this is a sufficient performance measure. In flow-carrying network models, however, the performance measure of the system also depends on the rate of resource flow rather than just the existence of an active path. This reachability module broadly aims to answer the network query of "is the network connected?" However, in networks that also model flow, knowing the likelihood of a signal arriving does not necessarily answer the query of "how much gets through".

Consider a resource distribution network where all source-to-sink paths satisfy the reachability survival threshold. Despite the network being considered connected, deliverable throughput at target nodes can still fall well short of demand if it enters a degraded capacity state. In flow-carrying networks, computing reachability alone cannot fully distinguish between a network with reachability but low capacity and one with reachability plus plenty of spare capacity at target nodes.

Using the fixed edge capacities stored on the extracted graph object in the InputProcessing step, the network flow toolkit consists of a collection of classical max-flow-based modules intended to support analysis of deliverable throughput, bottleneck structure, and the effect of structural or parametric degradation in capacitated DAG models. Specifically, given a capacitated directed acyclic graph as input, the toolkit is designed to address the following analysis objectives:

- Q1.** What maximum throughput can the network deliver under current capacities?
- Q2.** Which cut sets form the binding bottlenecks on that throughput?
- Q3.** Which component failures or capacity perturbations most reduce deliverable flow?
- Q4.** What degradation or upgrade threshold is required to maintain a target demand level?
- Q5.** How much edge and node redundancy exists in path-disjoint terms?
- Q6.** Which components are structural single points of failure?

0.2 Mathematical Model and Network Flow Semantics

Let

$$G = (V, E)$$

be a directed acyclic graph with edge-capacity function

$$c : E \rightarrow \mathbb{R}_{\geq 0},$$

where c_{uv} is the maximum admissible flow on edge $(u, v) \in E$. When node capacities are modeled, let

$$b : V \rightarrow \mathbb{R}_{\geq 0}.$$

Here b_v denotes the maximum admissible through-flow at node v . Define the flow variable

$$f : E \rightarrow \mathbb{R}_{\geq 0},$$

with $f_{uv} := f((u, v))$ for each edge $(u, v) \in E$. For node v , define

$$\delta^+(v) := \{(v, w) \in E\}, \quad \delta^-(v) := \{(u, v) \in E\}$$

as the outgoing and incoming edge sets, respectively. For a capacitated network model G with designated source and sink nodes $s, t \in V$, the primary objective is to compute the maximum deliverable throughput from s to t , subject to edge capacities c_{uv} for $(u, v) \in E$ and, when modelled, node through-flow bounds b_v for $v \in V$.

First consider a base-case network: a single-source single-sink capacitated DAG, shown in Fig. ???. Node s is the source, node t is the sink, and intermediate nodes relay flow under capacity constraints. The network performance measure must consider and quantify deliverable throughput from s to t , not just s - t path existence.

For this base case, let $f : E \rightarrow \mathbb{R}_{\geq 0}$ and $F \in \mathbb{R}_{\geq 0}$ be decision variables, where F is the total delivered flow from s to t . The optimization model is

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \tag{1}$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \tag{2}$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \tag{3}$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \tag{4}$$

Taking F^* to be the optimal (maximum feasible) value of F . Then if the sink node t has a required demand level D_t , the flow module quantifies service adequacy through

figures/flow_chapter/toy_siso_symbolic.pdf

Figure 1: Minimal single-source single-sink capacitated DAG network.

the throughput margin

$$\Delta_t := F^* - D_t,$$

where $\Delta_t \geq 0$ means that the network can meet demand with residual margin, and $\Delta_t < 0$ indicates a throughput shortfall of $|\Delta_t|$.

Multi-Source Multi-Sink Reduction

Let the source set be $S \subseteq V$ and the sink set be $T \subseteq V$, with $|S| \geq 1$ and $|T| \geq 1$. Figure ??a shows the original multi-source multi-sink network.

By introducing a super-source s^* and a super-sink t^* , as shown in Fig. ??b, the reduced graph is

$$V' = V \cup \{s^*, t^*\},$$

$$E' = E \cup \{(s^*, s) : s \in S\} \cup \{(t, t^*) : t \in T\}.$$

The connector capacities are set to non-binding values (implemented as Inf in the solver):

$$c_{s^*s} = +\infty \quad \forall s \in S, \quad c_{tt^*} = +\infty \quad \forall t \in T.$$

Proposition 1 (Exactness of the super-terminal reduction). *The optimal value of the*

figures/flow_chapter/msss_original.pdf

figures/flow_chapter/msss_super_reduction.pdf

(a) Original multi-source multi-sink network.

(b) Equivalent super-source/super-sink reduction.

Figure 2: Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.

multi-source multi-sink max-flow problem on G is equal to the optimal value of the single-source single-sink max-flow problem on the reduced graph $G' = (V', E')$ with terminals s^ and t^* .*

Proof. Let f be any feasible flow for the original multi-terminal problem with total value F .

We may define an augmented flow f' on G' by retaining the values of f on all original edges, setting

$$f'_{s^*s} := \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \quad \forall s \in S,$$

and

$$f'_{tt^*} := \sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \quad \forall t \in T.$$

Since the connector edges are guaranteed to be non-binding, then all capacity constraints from original network remain satisfied. Naturally, flow conservation holds at every original transit node, while the net outflow from s^* and the net inflow to t^* are both equal to F . Therefore f' is a feasible s^* - t^* flow of value F .

Conversely, let f' be any feasible s^* - t^* flow on G' : then, restricting f' to the original edge set E gives a flow f on G . Again, conservation on all nodes in $V \setminus (S \cup T)$ is inherited directly from the reduced problem, and the super-terminal connectors account exactly for aggregate injection at the sources and aggregate withdrawal at the sinks. Therefore f is feasible for the original multi-terminal problem and has the same total value as f' . Since feasible flows map in both directions without changing objective value, the two optimal

values are equal. $\square$

Thus, solving the reduced super-source/super-sink problem is equivalent to solving the original multi-terminal problem, which is:

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (5)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus (S \cup T), \quad (6)$$

$$\sum_{s \in S} \left(\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \right) = F, \quad (7)$$

$$\sum_{t \in T} \left(\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \right) = F. \quad (8)$$

Node-Capacitated Extension via Node Splitting

The Information Propagation Framework supports both node and edge network meta-data. For capacity analysis, this allows for modelling explicit throughput constraints on nodes themselves. As shown in Figure ??, this work adopts a node-splitting approach where each constrained node $v \in V_c$ is replaced by an input node v^- and an output node v^+ , connected by an internal edge:

$$(v^-, v^+), \quad c_{v^-v^+} = b_v.$$

The original incoming edges of v are redirected to v^- , and every original outgoing edge is emitted from v^+ .

This extends the edge-capacitated flow model with

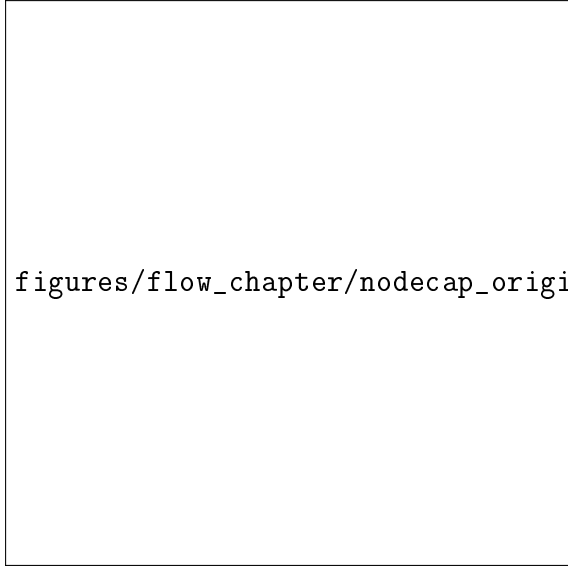
$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (9)$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c. \quad (10)$$

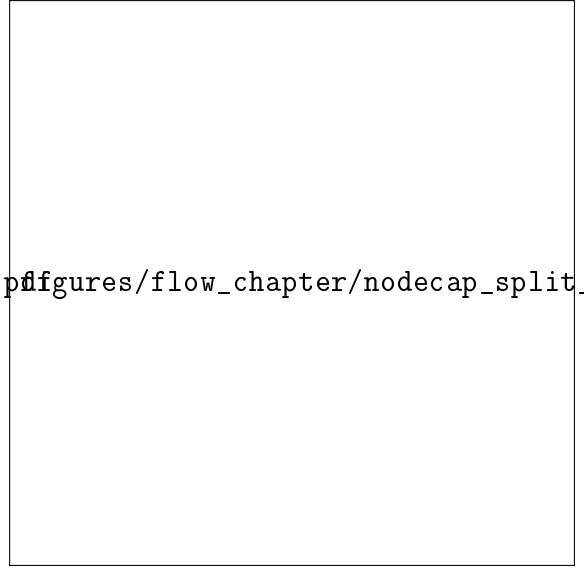
Here $V_c \subseteq V$ may include source and sink nodes when terminal throughput limits are part of the model.

The optimization model then becomes

$$\max F$$



(a) Original network with edge and node capacities.



(b) Equivalent node-split edge-capacitated network.

Figure 3: Node-splitting transformation.

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (11)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \quad (12)$$

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (13)$$

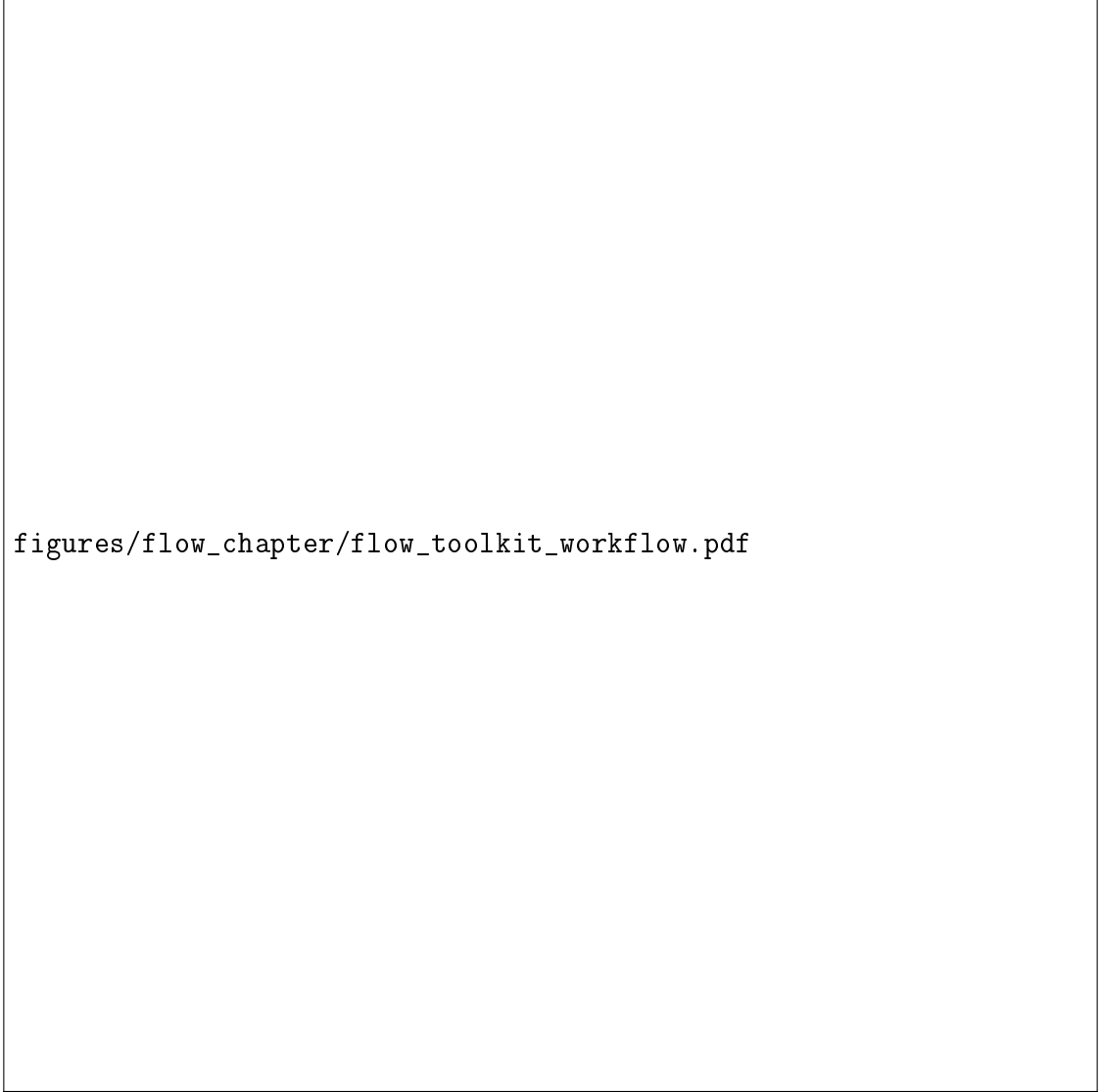
$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (14)$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \quad (15)$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \quad (16)$$

0.3 Available Modules

Where required, the flow toolkit applies the super-terminal reduction automatically during the baseline flow solve, and the node-capacitated analysis invokes node splitting internally whenever node-capacity data are supplied. Public outputs are then remapped back to the original graph and original source/sink sets, so edge flows, sink allocations, and cut-side node sets remain interpretable in the user's native network model.



figures/flow_chapter/flow_toolkit_workflow.pdf

Figure 4: Runtime workflow for the flow-capacity toolkit with available modules

0.4 Base Flow Module

Motivating question: “How much flow can the network sustain, and what limits it?”

Inspired by Julia’s `Graphs.jl` and Python’s `NetworkX` flow analysis packages, the flow engine supports three solver variants to compute F^* for a given capacitated network G , summarised in Table ??.

Solver	Complexity	Best Fit
<code>:edmonds_karp</code>	$O(VE^2)$	Small networks
<code>:dinic</code>	$O(V^2E)$	Larger layered networks
<code>:push_relabel</code>	$O(V^2E) - O(V^3)^a$	Dense, high fan-out networks

^a Upper bound depends on node selection strategy; this solver does not enforce a highest-label invariant.

Table 1: Max-flow solver comparison.

0.4.1 Max-Flow Min-Cut Solvers

The intuition behind Ford-Fulkerson’s [?, ?] Max-Flow Min-Cut (MCMF) theorem is that given a capacitated network, the maximum permissible flow through the network is restricted by the smallest blocking bottleneck set of edges. That is, the maximum possible flow from $s \rightarrow t$ given G is equal to the sum of flow across the min-cut.

Definition 1 (Residual Graph). *Given a capacitated network $G = (V, E)$ with capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$ and a feasible flow f , the residual graph $G_f = (V, E_f)$ is defined over the same node set V with residual edge set*

$$E_f = \{(u, v) \in E : c_{uv} - f_{uv} > 0\} \cup \{(v, u) : (u, v) \in E, f_{uv} > 0\},$$

where the residual capacity on each edge is given by

$$r_{uv} = \begin{cases} c_{uv} - f_{uv} & (u, v) \in E \quad (\text{forward residual}), \\ f_{vu} & (v, u) \in E \quad (\text{backward residual}). \end{cases}$$

A forward residual edge (u, v) represents spare capacity on an original edge; a backward residual edge (v, u) represents flow on (u, v) that can be cancelled. An augmenting path P in G_f is then a directed path from s to t along which $r_{uv} > 0$ for every edge $(u, v) \in P$.

The Ford-Fulkerson-based solvers work by iteratively finding and pushing flow along augmenting paths and generally follow these steps:

1. Initialise a working residual graph with zero flow.
2. Find an augmenting path P from source to sink.
3. Compute the residual (spare) capacity on all edges in P .
4. Push as much flow as possible along P , the maximum of which is limited by the edge with the least spare capacity on P .
5. Update the working graph.
6. Repeat from step 2 until no more augmenting paths remain.
7. Identify the min-cut from the reachable/non-reachable partitioning in the final residual graph.

As is evident from the algorithm steps described above, most of the computation time is spent repeating four main lookups:

- Capacity on any edge $u-v$
- Current flow on any edge $u-v$
- Outgoing edges from any node u for graph search expansion and forward residual updates
- Incoming edges to any node u for backward residual updates and inflow queries

The graph object from the input processing module already stores this information as part of the network structure and therefore does not need to be recomputed on every call. Both Ford-Folkerson-based solvers use this lookup structure but differ at step 2 in how the augmenting path is found.

Edmonds-Karp. The Edmonds-Karp solver uses a repeated breadth-first search (BFS) to find the shortest augmenting path in the residual graph. However, since each iteration requires an explicit path reconstruction via a stored parent map, the per-iteration overhead is higher than the Dinic variant, and the $O(VE^2)$ bound makes it poorly suited for larger and denser networks. It is therefore best reserved for smaller networks where simplicity and interpretability are preferred over performance.

Dinic. To avoid the one-path-at-a-time limitation of the Edmonds-Karp method, the Dinic solver alternates between level graph construction via BFS and a blocking-flow depth-first search. The level graph restricts traversal to edges $u \rightarrow v$ where $\text{level}[v] = \text{level}[u] + 1$, reducing the search space to a layered subgraph per phase. Furthermore, to avoid the $O(E)$ per-path rescanning cost of naive DFS, this variant uses a pointer cursor `ptr[u]` to track each node’s position in its adjacency list, permanently skipping exhausted edges.

The Dinic solver is the best fit for most hierarchical DAG networks and is the framework’s default when no user-specified option is selected. However, on very dense graphs with $O(V^2)$ edges and high structural redundancy, the per-phase BFS reconstruction becomes costly, and the blocking flow DFS must still handle a larger fraction of edges despite the pointer optimisation. In these cases, the Push-Relabel solver is recommended, as it operates locally without global path searches.

0.4.2 Push-Relabel Solver

Unlike the two path-based solvers, the Push-Relabel (PR) variant does not search for augmenting paths at all. This solver works by resolving excess flow locally from an initialised pre-flow. Here, instead of a residual graph per phase, a merged neighbourhood list is built once over the augmented network and shared across all push and relabel operations. The algorithm proceeds as follows:

1. Initialise $\text{height}[s^*] = |V|$, push maximum admissible flow from the super-source to its neighbours, and track $\text{excess}[v]$ as inflow minus outflow at each node.
2. While active nodes ($\text{excess}[u] > 0$) exist:
 - (a) Attempt to push flow from u to neighbour v where $\text{height}[u] = \text{height}[v] + 1$.
 - (b) If no valid push exists, relabel u by raising its height to $\min_v \text{height}[v] + 1$ over all reachable neighbours.
3. Terminate when no excess remains outside s^* and t^* .
4. Compute the min-cut by traversing the final flow state from s^* .

0.5 Network Flow Diagnostics

0.5.1 Min-Cut Utility Module

Motivating question: *“How constrained is the bottleneck, and in how many equivalent ways?”*

The flow module solvers all compute the max flow F^* by constructing a representative valid min-cut S^* in the given network G . This output by itself neither indicates nor guarantees that S^* is the only possible min-cut set in G .

This distinction matters in practice for some engineering decisions. Infrastructure systems are designed with robustness and redundancy in mind. In these networks it is more common to expect multiple structurally distinct S/T partitions of V that all have the same max flow F^* value.

While the baseline max flow value directly addresses the first analysis objective, the utility module exploits the already-generated min-cut and residual graph to build the extended partition S^{**} and compute the full space of valid min-cuts.

Let $R_T \subseteq V$ be the set of nodes reachable via backward traversal from t^* in G_f .

Definition 2 (Extended Source Partition). *The extended source-side region is*

$$S^{**} = V \setminus R_T,$$

restricted to original graph nodes. Any node outside R_T cannot reach the sink under the current flow state, so absorbing it into the source partition never increases cut capacity. Consequently $S^ \subseteq S^{**}$.*

Definition 3 (Free Zone). *Given the extended partition, the free zone is*

$$\mathcal{F} = S^{**} \setminus S^*.$$

Nodes in $\mathcal{F}$ can be assigned to either side of a minimum cut without affecting its capacity. Every minimum cut of G under f^ takes the form $S = S^* \cup R$ for some $R \subseteq \mathcal{F}$.*

The free zone size $|\mathcal{F}|$ itself determines the degeneracy of the bottleneck structure. Specifically, when $|\mathcal{F}| = 0$ the minimum cut computed from the flow solver is unique and S^* is the only valid partition. For $|\mathcal{F}| > 0$ there are exactly $2^{|\mathcal{F}|}$ equivalent minimum cuts, meaning the cut boundary is not pinned to a single partition.

Both the extended partition S^{**} and free zone $\mathcal{F}$ allow for a more complete picture of the bottleneck edge structure beyond what the representative cut alone provides. It then follows that two distinct edge sets can be derived directly from the lattice.

The set of edges appearing in at least one minimum cut is

$$E_{\text{some}} = \{(u, v) \in E : f_{uv}^* = c_{uv}, u \in S^{**}, v \notin S^*\},$$

and the set appearing in every minimum cut is

$$E_{\text{every}} = \{(u, v) \in E : f_{uv}^* = c_{uv}, u \in S^*, v \in T^{**}\},$$

where $T^{**} = V \setminus S^{**}$, and $E_{\text{every}} \subseteq E_{\text{some}}$ holds by construction.

Min-Cut Enumerator

Given the residual lattice, the enumerator function in this utility module iterates over subsets $R \subseteq \mathcal{F}$ in binary counting order. Since each subset maps to a distinct minimum cut $S^* \cup R$, every enumerated cut is guaranteed to be valid by lattice membership. The function exposes a `cut_limit` parameter such that: if $2^{|\mathcal{F}|} \leq \text{cut_limit}$ all minimum cuts are returned and `is_complete = true`; otherwise, the first `cut_limit` cuts are returned with `is_complete = false`. This can be necessary in highly symmetric networks, where $|\mathcal{F}|$ can be large, making full enumeration intractable.

0.5.2 Structure Utility Module

Motivating question: “Where is the network structurally fragile?”

Designed to inform routing and redundancy decisions, the structural module identifies components that are fragile purely due to their position in G , regardless of any capacity assignment.

Understanding the route-level structure of G is a prerequisite for any path-level fragility or flow-distribution analysis. To this end, the module exposes a base path enumerator function that performs a depth-first search on the topologically sorted iteration set derived from the `InputProcessingModule`. Since G is a DAG, exact enumeration is always finite; however, highly redundant networks can admit exponentially many routes, so a `path_limit` parameter bounds the number of enumerated paths and returns a representative sample rather than the complete set $\mathcal{P}$.

Flow Decomposition. Two networks can share identical topology and the same F^* while concentrating flow through entirely different routes. During design decisions where multiple configurations are evaluated and per-route visibility is needed, this decomposition exposes how f^* is distributed across $\mathcal{P}$.

Given an optimal flow f^* and a set of flow-agnostic paths $\mathcal{P}$, for each path $P = (v_0, v_1, \dots, v_k) \in \mathcal{P}$, the bottleneck flow contribution is the minimum flow on any edge along P :

$$\phi(P) = \min_i f_{v_i v_{i+1}}^*,$$

Thus, the edge meeting this minimum must be the path’s local bottleneck. Paths are sorted in descending order of $\phi(P)$ as a ranked decomposition of f^* into per-route contributions. Where the min-cut module’s ranking informs global capacity upgrade decisions, this ranking informs targeted interventions at the route level. In a network supplying two distinct demand nodes via separate routes, the min-cut module points to the tightest

cut edge across the whole network, whereas the decomposition identifies independently which component is constraining delivery to each demand node.

Structural Single Point of Failure Nodes.

Definition 4 (Structural Single Point of Failure). *A node $v \in V \setminus (S \cup T)$ is a SPOF node if $F^*(G \setminus \{v\}) = 0$, where removing v also removes all incident edges. A SPOF node is one through which every source-to-sink path must pass; its removal alone disconnects the network regardless of any capacity assignment.*

The candidate set is restricted to nodes lying on at least one s - t path,

$$\mathcal{C} = (\text{fwd}(S) \cap \text{bwd}(T)) \setminus (S \cup T),$$

where $\text{fwd}(S)$ and $\text{bwd}(T)$ are the sets of nodes forward-reachable from S and backward-reachable from T in G respectively. Nodes outside $\mathcal{C}$ lie on no s - t path and cannot be SPOF nodes. For each $v \in \mathcal{C}$, a BFS is run on $G[V \setminus \{v\}]$; if no sink in T is reachable from any source in S , v is recorded as a SPOF node. The function requires one BFS per candidate and runs in $O(|\mathcal{C}| \cdot (|V| + |E|))$ time.

Noted that in a node-capacitated network, a node appearing in the min-cut is not necessarily a SPOF node in this structural sense. A capacity-constrained node enters the cut-space by way of its split edges $(v_{\text{in}}, v_{\text{out}})$ with capacity b_v and is not necessarily a structural choke-point in G . In these networks, a node can simultaneously be a flow-limiting bottleneck and a SPOF node.

0.5.3 Flow Sensitivity Module

Motivating question: *“Which edges most constrain deliverable throughput, and by how much?”*

At optimality, the flow f^* partitions the edge set into two classes: edges carrying strictly less than their capacity, $f_{uv}^* < c_{uv}$, and edges that are fully utilised,

$$f_{uv}^* = c_{uv}.$$

The latter are referred to as *saturated edges* in this work. By the max-flow min-cut theorem, every minimum cut set is made up only of saturated edges; therefore, any augmenting path from S to T must cross at least one saturated edge. Since the binding constraint lies elsewhere, *unsaturated edges* carry slack and are transparent to perturbation — increasing or decreasing their capacity by a small enough amount has little to no effect on F^* . The Flow Sensitivity module focuses only on saturated edges, since they

collectively form the capacity-critical boundary of the network. This module exposes the following three functions.

Critical Edge Ranking. An obvious measure of an edge’s failure impact is how much the max flow drops when that edge is disconnected from the network, referred to as edge criticality in this module. The function re-runs the flow solver on a modified network for each candidate. For each saturated edge (u, v) , the critical impact is:

$$\Delta_{uv}^* = F^*(G, c) - F^*\left(G, c_0^{(uv)}\right),$$

where $c_0^{(uv)}$ denotes the capacity function with c_{uv} set to zero and all other capacities unchanged. The function returns the edges in descending order of Δ_{uv}^* .

Marginal Capacity Values. Similar to the critical edge ranker, the function is scoped to saturated edges. For each saturated edge (u, v) , it quantifies the incremental gain in F^* per unit of additional capacity,

$$\mu_{uv} = \frac{F^*(G, c_\delta^{(uv)}) - F^*(G, c)}{\delta},$$

where $c_\delta^{(uv)}$ increases c_{uv} by δ with all other capacities unchanged.

In highly redundant networks, bypass routes often exist that can absorb rerouted flow and upgrade priorities are not always clear. In these systems, an edge can rank high for failure impact but have a lower marginal value. The critical edge function identifies which edges are most damaging to lose while the marginal capacity function complements this by identifying which ones are most worth upgrading.

Marginal Range. Both Δ^* and μ are snapshot evaluations relative to the current capacity assignment c . Depending on the network structure, an edge with a low μ under the current inputs may become highly influential under different capacity values. This Marginal Range function evaluates the total range of influence a single edge (u, v) can exert on F^* :

$$\mathcal{B}_{uv} = F^*(G, c_\infty^{(uv)}) - F^*\left(G, c_0^{(uv)}\right),$$

where $c_\infty^{(uv)}$ sets $c_{uv} = \infty$ and $c_0^{(uv)}$ sets $c_{uv} = 0$, and all other capacities fixed. This is function is analogous to the classical Birnbaum importance measure $I_B(i) = \partial h(\mathbf{p})/\partial p_i$ (see [?]). However, the analogy is conceptual since $\mathcal{B}_{uv}$ is a deterministic throughput range, not a probability-weighted sensitivity.

0.5.4 Failure Impact Module

Motivating question: “How is deliverable throughput affected when components fail or degrade?”

The failure impact module models outright component failures and degraded operating states.

Single and Combined Edge Failures. For a single failed edge (u, v) , the throughput impact is:

$$\Delta_{uv} = F^*(G, c) - F^*\left(G, c_0^{(uv)}\right),$$

where $c_0^{(uv)}$ sets $c_{uv} = 0$ with all other capacities unchanged and candidate edges are restricted to E_{some} .

Real failure events are not always isolated to a single component. A storm or substation fault in a power network can take down multiple edges simultaneously. Since the interaction between affected edges may open or close re-routing options, the combined impact of k simultaneous failures is not in general the sum of their individual Δ_{uv} impacts.

The k -edge failure function generalises the single failure case to any set of k edges removed simultaneously. For a combination $\mathcal{K} \subseteq E_{\text{some}}$ with $|\mathcal{K}| = k$, the combined impact is:

$$\Delta_{\mathcal{K}} = F^*(G, c) - F^*\left(G, c_0^{(\mathcal{K})}\right),$$

where $c_0^{(\mathcal{K})}$ sets $c_{uv} = 0$ for all $(u, v) \in \mathcal{K}$. All $\binom{|E_{\text{some}}|}{k}$ combinations require one re-solve each. In practice, because $\binom{|E_{\text{some}}|}{k}$ grows rapidly with k , this function is most tractable for $k = 2$ or $k = 3$ depending on the size and density of the network. The function returns a complete ranking of k -edge failure scenarios by throughput impact.

Capacity Degradation Scenarios. Infrastructure components don’t always fail completely. Partial blockage or degraded components are more likely to present as a reduction in c_{uv} rather than complete loss. Additionally, a domain expert working across multiple network states may want to compare the effect of varying input capacities and operating conditions. This function is intended to be used to efficiently compute the full F^* picture across all scenarios without needing to re-run the input processing or baseline solve steps for each. The degradation function accepts either an explicit override dictionary specifying new capacities for individual edges, or a uniform scale factor $\alpha \in [0, 1]$ applied to all finite capacities,

$$c'_{uv} = \alpha \cdot c_{uv}, \quad \forall (u, v) \in E.$$

0.5.5 Parametric Threshold Module

Motivating question: “How much capacity change is needed to maintain or breach a target service level?”

The previous modules treat edge capacities as fixed and analyse the network’s response to perturbations around them. The threshold module inverts this perspective. Given a required demand level $D_t \leq F^*$ to be met, it determines what capacity each edge must have for the network to meet that target delivery level. This makes it a planning and design tool rather than a diagnostic one.

This module exploits a specific structural property of F^* : it is piecewise linear in any single edge capacity c_e with all others fixed. Logically, each linear segment maps to a fixed min-cut partition S^* , over which the threshold satisfying $F^*(c_e) = D_t$ can be recovered analytically. Therefore, breakpoints must occur exactly where the min-cut partition shifts, i.e. where a change in c_e causes a different set of edges to become the binding bottleneck.

Degradation Threshold. For a target edge (u, v) and a required flow level D_t , the degradation threshold $c_{uv}^\dagger$ is the minimum capacity the edge must retain for F^* to remain at least D_t :

$$c_{uv}^\dagger = \inf\{c \in [0, c_{uv}] : F^*(G, c_e^{(uv)}) \geq D_t\},$$

where $c_e^{(uv)}$ denotes the capacity function with c_{uv} replaced by c . The degradation margin $c_{uv} - c_{uv}^\dagger$ is the slack available before the service level is breached. The module ranks edges by ascending degradation margin, so the most vulnerable edges appear first. A small margin can be interpreted as the edge is close to becoming a binding constraint on D_t , while a large margin means the edge can absorb substantial capacity loss without service impact.

The threshold is located by a partition-change recursion. Starting from the known endpoints $c_e = 0$ and $c_e = c_{uv}$, the algorithm evaluates F^* at the midpoint and compares the resulting min-cut partition to those at the endpoints. If both endpoints share the same partition, the segment is linear and the threshold is solved analytically. If the partitions differ, the interval contains a breakpoint and the recursion descends into the sub-interval containing D_t . This continues until the threshold is isolated to a partition-constant segment, where it is computed exactly. The result is exactly capacity value at which F^* crosses D_t .

Two degenerate cases are handled explicitly. If $F^*(c_e = 0) \geq D_t$, the edge can degrade to zero without breaching the target. If $F^*(c_e = c_{uv}) < D_t$, the target is unachievable even at the current capacity, meaning the bottleneck lies entirely elsewhere and this edge cannot help.

Upgrade Threshold. If $F^* < D_t$ under the current capacity assignment, the upgrade threshold is the minimum capacity (u, v) must have for F^* to first reach D_t . Since the required capacity may exceed c_{uv} , the upper search bound is not known in advance. The algorithm uses a doubling search to bracket the threshold interval, repeatedly doubling the trial capacity until $F^* \geq D_t$, then applies the same partition-change recursion to locate it exactly.

An edge case arises when the edge is not on any minimum cut: expanding its capacity cannot raise F^* regardless of how large c_{uv} becomes. The doubling search detects this when F^* stops increasing before reaching D_t and returns `upgrade_ineffective = true`, indicating that reaching D_t requires addressing a different bottleneck.

When no explicit target is supplied, the module defaults to $D_t = 0.9 \cdot F^*$.

0.6 Exactness, Validation, and Scope

Exactness as used in this chapter is not the same as global combinatorial exhaustiveness. Where candidate-set pruning is applied, exactness describes the results on evaluated candidates only, not globally across all possible edge subsets or perturbation scenarios.

0.6.1 Post-solve validation

Every public solver call with validation enabled applies three post-solve consistency checks. (i) Capacity feasibility: all edge flows satisfy $0 \leq f(u, v) \leq c(u, v)$. (ii) Node-wise conservation: $\sum_{u:(u,v) \in E} f(u, v) = \sum_{w:(v,w) \in E} f(v, w)$ at all internal nodes. (iii) Max-flow/min-cut consistency: the solved flow value and the extracted cut capacity agree within numerical tolerance. These confirm primal feasibility and dual optimality consistency before derived analyses are interpreted.

0.6.2 Scope of results

Four main scope boundaries apply throughout this chapter. First, the sensitivity and failure-impact modules use candidate sets drawn from the baseline min-cut structure; results are exact on evaluated candidates, and global exhaustiveness over all edges or all combinatorial subsets requires explicit full-space evaluation. Second, the capacity toolkit does not compute $P(\text{performance} \geq T)$ or any probability-weighted reliability index; uncertainty models for random capacity failures are handled in subsequent chapters. Third, all analyses operate on a single fixed-capacity instance; time-varying degradation, adaptive rerouting, and multi-period operational models are outside the scope of this layer. Finally, for complete throughput loss, minimum cuts are the minimal failure modes; for nonzero service targets, the relevant boundary is characterised by threshold margins $c_{uv} - c_{uv}^\dagger$.

0.6.3 Computational Complexity

The total computational cost of any derived analysis is the product of the per-solve cost of the chosen max-flow algorithm and the number of solver calls required. Repeated-solve tasks scale with solve count, which is the dominant cost driver in practice. Per-solve costs are summarised in Table ??.

	Module	Function	Operation	Complexity
<i>Base Solver</i>				
Base	Edmonds-Karp		BFS augmenting path	$O(VE^2)$
Base	Dinic		Level graph + blocking	$O(V^2E)$
Base	Push-Relabel		Preflow relabeling	$O(V^3)$
Node-Cap	Split transformation		Graph transform	$O(V + E)$
Node-Cap	Solve (split)		Solver on $2V, 2E$	$O(T_{\text{solver}})$
<i>Min-Cut Utility</i>				
Min-Cut	cut_edges		Extract partition	$O(E)$
Min-Cut	edges_some		BFS + filter	$O(E + T_s)$
Min-Cut	edges_every		Reachability	$O(V + E)$
Min-Cut	enumerate		Lattice ($2^{ \mathcal{F} }$)	$O(2^{ \mathcal{F} } \cdot E)^1$
<i>Structural</i>				
Struct	spof_nodes		BFS per candidate	$O(\mathcal{C} (V + E))$
Struct	enumerate_paths		DFS on DAG	$O(\mathcal{P})^2$
Struct	path_flow		Per-path scan	$O(\mathcal{P} \ell)$
Struct	redundancy		Menger (unit cap)	$O(E_c T_s)$
<i>Flow Sensitivity</i>				
Sens	critical_edges		Zero-cap solves	$O(E_{\text{sat}} T_s)$
Sens	marginal_capacity		$+\delta$ solves	$O(E_{\text{sat}} T_s)$
Sens	marginal_range		Range evals	$O(E_{\text{some}} T_s)$
<i>Failure Impact</i>				
Fail	single_edge		Per-edge solves	$O(E_{\text{some}} T_s)$
Fail	k_edge		Per-combination	$O(\binom{ E_s }{k}T_s)^3$
Fail	degradation		Per-scenario	$O(S T_s)$
<i>Parametric Threshold</i>				
Param	degrade		Partition recursion	$O(\lg(c/\varepsilon)T_s)$
Param	upgrade		Doubling + recurse	$O(\lg^2(c/\varepsilon)T_s)$
Param	all_degrade		Per edge	$O(E_c \lg(c/\varepsilon)T_s)$

Table 2: Per-task computational complexity for the flow capacity toolkit. T_s denotes the cost of a single max-flow solve under the chosen algorithm.

0.7 Network Flow Output Structure

Field	Module	Description
<code>max_flow</code>	Base Solver	Maximum deliverable throughput F^* from source set to sink set.
<code>flow</code>	Base Solver	Final flow on each original edge.
<code>residual_capacity</code>	Base Solver	Spare capacity on each original edge after the solve.
<code>node_flow</code>	Base Solver	Through-flow at each node: outflow at sources, inflow at sinks, conserved flow at transit nodes.
<code>sink_flow</code>	Base Solver	Delivered flow broken down by individual sink.
<code>mincut_S, mincut_T</code>	Base Solver	Source- and sink-side min-cut node partitions.
<code>mincut_capacity</code>	Base Solver	Capacity of the reported minimum cut.
<code>is_unbounded</code>	Base Solver	Whether unbounded max-flow was detected.
<code>saturated_edges</code>	Base Solver	Edges carrying flow at full capacity $f_{uv}^* = c_{uv}$; forms the candidate set for sensitivity and failure impact modules.
<code>node_cap_max_flow</code>	Node-Cap Base Solver	Maximum deliverable throughput F^* under node capacity constraints.
<code>node_cap_flow_drop</code>	Node-Cap Base Solver	Reduction in F^* relative to the edge-only baseline.
<code>saturated_nodes</code>	Node-Cap Base Solver	Nodes whose through-flow reaches the node capacity bound b_v under the node-capacitated solve.
<code>mincut_S_star_star</code>	Min-Cut Utility	S^{**} : extended source partition; nodes that cannot reach the sink in the residual graph.

Field	Module	Description
<code>free_zone_size</code>	Min-Cut Utility	$ \mathcal{F} $: number of free-zone nodes assignable to either partition without affecting cut capacity. Exactly $2^{ \mathcal{F} }$ equivalent minimum cuts exist.
<code>edges_in_some</code>	Min-Cut Utility	E_{some} : edges appearing in at least one minimum cut.
<code>edges_in_every</code>	Min-Cut Utility	E_{every} : edges appearing in every minimum cut; structural single points of failure.
<code>mincut_enumeration</code>	Min-Cut Utility	Bounded enumeration of all valid minimum cuts in deterministic order, with completion flag <code>is_complete</code> and count.
<code>spof_nodes</code>	Structure Utility	Structural single points of failure nodes: $v \in \mathcal{C}$ where every s - t path must pass through v .
<code>enumerated_paths</code>	Structure Utility	Source-to-sink paths $\mathcal{P}$, bounded by <code>path_limit</code> .
<code>path_contributions</code>	Structure Utility	Per-path bottleneck flow $\phi(P) = \min_i f_{v_i v_{i+1}}^*$ ranked in descending order.
<code>critical_edges</code>	Flow Sensitivity	Saturated edges ranked by max-flow drop Δ_{uv}^* when set to zero capacity, in descending order.
<code>marginal_capacity</code>	Flow Sensitivity	Saturated edges mapped to incremental max-flow gain μ_{uv} per unit capacity increase.
<code>marginal_range</code>	Flow Sensitivity	Total range of influence $\mathcal{B}_{uv} = F^*(G, c_\infty^{(uv)}) - F^*(G, c_0^{(uv)})$ for each edge.
<code>fail_min_cut_edges</code>	Failure Impact	Representative min-cut edge set: all edges (u, v) with $u \in S^*$ and $v \in T^*$.
<code>single_edge_fail</code>	Failure Impact	Edges in E_{some} ranked by throughput drop Δ_{uv} when set to zero capacity, in descending order.
<code>k_edge_fail</code>	Failure Impact	All k -edge combinations from E_{some} ranked by combined impact $\Delta_{\mathcal{K}}$, bounded by <code>combination_limit</code> .

Field	Module	Description
degrade_results	Failure Impact	Max-flow and saturated edges for each capacity degradation scenario: either uniform scale factor α or edge-wise overrides.
degrade_thresholds	Parametric Threshold	For each candidate edge, the minimum capacity $c_{uv}^\dagger$ required to maintain target flow D_t , ranked by ascending degradation margin $c_{uv} - c_{uv}^\dagger$.
upgrade_thresholds	Parametric Threshold	For each candidate edge, the minimum capacity required for $F^\star$ to first reach D_t , with <code>upgrade_ineffective</code> flag where applicable.
param_target_flow	Parametric Threshold	Target service level D_t . Defaults to $0.9 \cdot F^\star$ if not specified.
param_baseline_flow	Parametric Threshold	Baseline max flow $F^\star$ for reference.

Table 3: Unified output schema for network flow capacity analysis across all modules.

0.8 Case Study: Synthetic Benchmark Network

0.8.1 Benchmark Network Design

To demonstrate the full range of the flow toolkit, a synthetic benchmark was generated. The network is a multi-source, multi-sink capacitated DAG with 43 nodes, 107 directed edges. The benchmark simulates a distribution system that propagates resources across geographically dispersed locations. As shown in Figure ??, it is structured as an 8-layer hierarchy:

- **Origin nodes** (blue): five source nodes, over-provisioned so the supply zone does not form the binding bottleneck.
- **Hub nodes** (green): eight intermediate relay nodes with high fan-out capacities.
- **Processing nodes** (red, node-capacitated) and **bypass nodes** (orange, node-capacitated): the processing corridor carries the bulk of flow; the bypass is a lightweight alternative with tighter node constraints.
- **Trunk and secondary nodes** (purple, brown): distribute flow through a re-converging structure toward the gateway layer.
- **Dispatch gateways** (light blue, node-capacitated): two gateways assigned equal edge capacity of 17 by design.
- **Distribution combiner** (pink): aggregates all gateway flow before fan-out to the sinks.
- **Sink nodes** (grey): seven demand nodes, each with incoming edge capacity 6.

Based on this layered structure, edge and node capacity assignments are chosen to ensure that binding bottlenecks emerge at specific layers under the different analysis methods. Edge capacities in Layers 1–5 are scaled by a factor of 2 relative to the gateway layer. In this way, the gateway layer governs F^* under the edge-only solution. Similarly, to compute a measurable reduction in F^* , the processing node capacities are set to bind to the node-capacitated solve.

To explore competing design alternatives under controlled conditions, two configurations of the same system are presented to evaluate the effect of feeder reinforcement (Figures ?? and ??). Both configurations of the network keep the same backbone topology, but represent different design objectives. Therefore, the comparative objective is not to maximize throughput alone, but to test whether robustness structure changes even when headline flow remains unchanged.

Config A proposes a symmetry-preserving design with balanced feeder-edge capacities and a limited direct bypass to sink 4. The capacity is set below the sink’s 6-unit limit such that the bypass delivers partial rather than full service. Config B’s asymmetric gateway structure is designed to produce non-trivial parametric upgrade thresholds and non-zero marginal-range values. This version has larger feeder capacities from secondary nodes into both gateways.

Table ?? summarises the key structural differences between the two. The expectation is that two designs may yield the same max-flow value while looking very different in terms of saturation pattern and min-cut landscape due to their differing feeder-capacity policies.

0.8.2 Results

From the base flow solver run, both configurations return $F^* = 37.0$, with `mincut_capacity` = 37.0, which is consistent with the expected maximum feasible flow for the benchmarked system. However, when optimizing for a demand target or comparing two network design approaches, F^* alone is not always enough to inform decisions. As shown in Table ??, the two networks still differ in structure: Configuration A has 41 saturated edges versus 35 in Configuration B, and it has 4 valid min-cuts versus 1 in Configuration B. This is also reflected in free-zone size (2 in A, 0 in B). The same throughput being produced by structurally different network constraints motivates the use of the extended analysis modules in the toolkit.

(a) Configuration A: symmetric gateway design. Blue edge $132 \rightarrow 140$ is the direct bypass to `sink_4`.

(b) Configuration B: orange edges show capacity-boosted secondary-to-gateway feeders, breaking gateway symmetry.

Figure 5: Flagship benchmark DAG (43 nodes, 107 edges, 8 layers). Node colours: blue = sources, green = hubs, red = constrained nodes, orange = bypass nodes, purple = trunks, brown = secondary, pink = combiner, grey = sinks.

Figure 6

Figure 7

Table 4: Configuration comparison summary.

Metric	Config A	Config B
F^*	37.0	37.0
saturated_edges	41	35
free_zone_size	2	0
min_cuts	4	1
SPOF_nodes	0	0
upgrade_ineffective (gateway edges)	true	false

Table 5: Top critical edges for Config A (non-zero deltas).

Rank	Label	ΔF^*	perturbed_flow
1	dispatch_gateway_west→distribution_combiner	17.0	20.0
2	dispatch_gateway_east→distribution_combiner	17.0	20.0
3	secondary_a→dispatch_gateway_west	4.0	33.0
4	secondary_c→dispatch_gateway_east	4.0	33.0
5	secondary_d→dispatch_gateway_west	4.0	33.0
6	secondary_f→dispatch_gateway_east	4.0	33.0
7	secondary_b→dispatch_gateway_west	3.0	34.0
8	secondary_b→dispatch_gateway_east	3.0	34.0
9	secondary_e→dispatch_gateway_west	3.0	34.0
10	secondary_e→dispatch_gateway_east	3.0	34.0
11	secondary_e→sink_4	3.0	34.0
12	secondary_c→dispatch_gateway_west	2.0	35.0
13	secondary_d→dispatch_gateway_east	2.0	35.0
14	secondary_a→dispatch_gateway_east	1.0	36.0
15	secondary_f→dispatch_gateway_west	1.0	36.0

Table 6: Min-cut enumeration for Config A.

Cut	Crossing edges
1	(128 → 134), (128 → 135), (129 → 134), (129 → 135), (130 → 134), (130 → 135), (131 → 134), (131 → 135), (132 → 134), (132 → 135), (132 → 140), (133 → 134), (133 → 135)
2	(128 → 135), (129 → 135), (130 → 135), (131 → 135), (132 → 135), (132 → 140), (133 → 135), (134 → 136)
3	(128 → 134), (129 → 134), (130 → 134), (131 → 134), (132 → 134), (132 → 140), (133 → 134), (135 → 136)
4	(132 → 140), (134 → 136), (135 → 136)

Figure 8

Bibliography

- [1] Ahuja, R.K., Magnanti, T.L., Orlin, J.B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ.
- [2] Birnbaum, Z.W. (1969). On the importance of different components in a multicomponent system. In P.R. Krishnaiah (Ed.), *Multivariate Analysis II*, pp. 581–592. Academic Press, New York. DOI: 10.21236/AD0670563.
- [3] Dinic, E.A. (1970). Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11, 1277–1280.
- [4] Edmonds, J., Karp, R.M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2), 248–264. DOI: 10.1145/321694.321699.
- [5] Ford, L.R., Fulkerson, D.R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 399–404. DOI: 10.4153/CJM-1956-045-5.
- [6] Ford, L.R., Fulkerson, D.R. (1962). *Flows in Networks*. Princeton University Press, Princeton, NJ.
- [7] Gallo, G., Grigoriadis, M.D., Tarjan, R.E. (1989). A fast parametric maximum flow algorithm and applications. *SIAM Journal on Computing*, 18(1), 30–55. DOI: 10.1137/0218003.
- [8] Goldberg, A.V., Tarjan, R.E. (1988). A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4), 921–940. DOI: 10.1145/48014.61051.
- [9] Hao, J.X., Orlin, J.B. (1994). A faster algorithm for finding the minimum cut in a directed graph. *Journal of Algorithms*, 17(3), 424–446. DOI: 10.1006/jagm.1994.1043.
- [10] Menger, K. (1927). Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10, 96–115. DOI: 10.4064/fm-10-1-96-115.
- [11] Picard, J.-C., Queyranne, M. (1982). A network flow solution to some nonlinear 0–1 programming problems, with applications to graph theory. *Networks*, 12(2), 141–159. DOI: 10.1002/net.3230120206.

- [12] Whitney, H. (1932). Congruent graphs and the connectivity of graphs. *American Journal of Mathematics*, 54(1), 150–168. DOI: 10.2307/2371086.